



Universidade Federal do Ceará
Centro de Tecnologia
Curso de Engenharia de Computação

Projeto 02 — Processamento Digital de Imagens

Dando Nitidez ao Invisível: Realce Laplaciano em Imagens
Digitais

Felipe Cordeiro de Sousa – 571941

Fortaleza, 23 de Outubro de 2025

Esse relatório referente ao projeto 2 da disciplina de Processamento Digital de Imagens (PDI) tem, por finalidade, ilustrar como funciona a implementação da técnica bem como a fundamentação matemática embasada para o uso da segunda derivada para aguçamento de imagens digitais ou Laplaciano. O presente trabalho fornece uma abordagem prática usando Python com pacotes especializados em manipulação de pixel, matemática e álgebra, como OpenCV e NumPy, conforme solicitado na disciplina de PDI.

1 Discussão Técnica

Nesse relatório, inicialmente será abordado os aparatos matemáticos necessários para que seja possível compreender a potencialidade do laplaciano discreto bem como o trecho de cada código usado para a replicação dos resultados obtidos.

1.1 Aspecto Matemático

Conforme demonstrado por Rosenfeld e Kak [Rosenfeld, 1976], o operador derivativo isotrópico mais simples é o laplaciano. Antes de partirmos para a formulação do laplaciano, é importante ressaltar o significado de filtros isotrópicos, são aqueles cujo a resposta depende da direção das descontinuidades (derivadas) da imagem a qual ela é sujeita ao filtro e são invariantes a rotação. Portanto, chegamos na formulação do laplaciano para uma função $f(x, y)$ de duas variáveis (imagem) é definida como:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (1)$$

Derivadas de qualquer ordem são operações lineares então, portanto, o laplaciano é um operador linear. A expressão dessa equação na forma discreta, é encontrando a derivada parcial de segunda ordem da função $f(x, y)$ na direção x e, em seguida analogamente para a direção y . Portanto, na direção x , temos:

$$\frac{\partial^2 f}{\partial x^2} = f(x+1, y) + f(x-1, y) - 2f(x, y) \quad (2)$$

E na direção y , temos:

$$\frac{\partial^2 f}{\partial y^2} = f(x, y+1) + f(x, y-1) - 2f(x, y) \quad (3)$$

Dessa forma, temos todos os termos da Equação 1 e, portanto, basta somarmos as duas Equações 2 e 3 e teremos a Equação 4 que define o laplaciano discreto de duas variáveis, veja:

$$\nabla^2 f = f(x, y+1) + f(x, y-1) + f(x+1, y) + f(x-1, y) - 4f(x, y) \quad (4)$$

Conforme descrito por Gonzalez e Woods, é importante levar em consideração qual definição de laplaciano será empregado dado cada caso, pois ora o laplaciano atua de uma forma ora de outra a depender do sinal da Equação 4 [Gonzalez, 2009]. Assim, a forma básica na qual utilizamos o laplaciano para o aguçamento de imagens é dado por:

$$g(x, y) = f(x, y) + c[\nabla^2 f(x, y)] \quad (5)$$

onde $f(x, y)$ e $g(x, y)$ são as imagens de entrada e aguçada, respectivamente, e c é uma constante geralmente considerada como 1 ou -1.

1.2 Implementação Prática

O algoritmo 1 implementado usando somente NymPY reflete os contextos matemáticos abordados anteriormente diante de uma perspectiva de código. A seguir serão comentados os dois trechos de códigos responsáveis pela geração da imagem de resultado 1. Inicialmente temos o algoritmo 1 conforme descrito abaixo:

```

1 LAPLACE4 = np.array([[0, 1, 0],
2                     [1,-4, 1],
3                     [0, 1, 0]], dtype=float)
4
5 LAPLACE8 = np.array([[1, 1, 1],
6                     [1,-8, 1],
7                     [1, 1, 1]], dtype=float)
8
9 def laplacian_numpy(img, laplace, pad_mode="reflect"):
10     """img: HxW ou HxWxC; retorna float64"""
11     img = np.asarray(img, dtype=float)
12     if img.ndim == 2: # grayscale
13         return _conv2_valid_pad(img, laplace, pad_mode)
14     elif img.ndim == 3: # RGB/RGBA
15         chans = [ _conv2_valid_pad(img[...,c], laplace, pad_mode) for c
16                 in range(img.shape[2]) ]
17         return np.stack(chans, axis=2)
18     else:
19         raise ValueError("Imagem deve ser 2D (HxW) ou 3D (HxWxC).")
20
21 def _conv2_valid_pad(gray, kernel, pad_mode):
22     kh, kw = kernel.shape
23     ph, pw = kh//2, kw//2
24     gp = np.pad(gray, ((ph,ph),(pw,pw)), mode=pad_mode)
25     # janelas (H, W, kh, kw)
26     win = sliding_window_view(gp, (kh, kw))
27     return np.tensordot(win, kernel, axes=((-1,-2),(1,0))) # mesma
28     forma de gray

```

Listing 1: Função que implementa o Laplaciano discreto (usando NumPY).

O algoritmo 1 cria dois kernels 3×3 , o primeiro o pixel central de um é -4 enquanto que suas diagonais são atribuídas a zero, o segundo com todos os pixels adjacentes ao central (-8) é 1. Ele será responsável por aplicar o realce na imagem, transladando a imagem com um dado passo definido. Em seguida garante que a imagem seja manipulada como vetor NumPY, explicitamente com o tipo float, e verificamos se a imagem está na escala de cinza ou colorida (RGB), a depender da verificação realiza suas devidas modificações e aplica a função de convolução 2d.

Para a função principal que implementa a ideia do laplaciano discreto, coletamos as informações de altura e largura do kernel, encontramos o preenchimento na largura e altura dividindo por 2 as informações coletadas do kernel e usamos como parâmetro na função *np.pad* e, após isso, usamos uma função altamente otimizada do NumPY para criar uma visualização da imagem preenchida (gp). Essa visão (win) representa todas as submatrizes 3×3 (as "janelas") sobre as quais o kernel deve ser aplicado e, por fim, fazemos uma convolução usando *np.tensordot*.

A partir do que vimos de implementação prática do laplaciano discreto em Python usando NumPY, é válido entendermos como a imagem deve ser pré-processada para que possa ser aplicada corretamente o laplaciano discreto.

```

1 # Normalizacao
2 img_moon_normalized = img_moon_array.astype(np.float32) / 255.0
3 img_moon_float = img_moon_array.astype(np.float32)
4
5 moon_image_laplacian_4 = laplacian_numpy(img_moon_normalized, laplace=
6     LAPLACE4) * 255.0 #np.float32

```

```

6 moon_image_laplacian_8 = laplacian_numpy(img_moon_normalized, laplace=
  LAPLACE8) * 255.0 #np.float32
7 moon_image_laplacian_adjust = img_moon_float - moon_image_laplacian_4
8 moon_image_laplacian_adjust_8 = img_moon_float - moon_image_laplacian_8
9
10 # Aplica o deslocamento de 128
11 moon_image_laplacian_4_deslocado = moon_image_laplacian_4 + 128
12
13 # Limitar (Clip) e Converter para 8-bit (uint8) para visualizacao, pois
  os valores podem estar fora da faixa [0, 255] apos a subtracao
14
15 # Clipa e converte para uint8 para exibicao (Figura C)
16 moon_image_laplacian_4_deslocado = np.clip(
  moon_image_laplacian_4_deslocado, 0, 255).astype(np.uint8)
17 moon_image_adjust_4 = np.clip(moon_image_laplacian_adjust, 0, 255).
  astype(np.uint8)
18 moon_image_laplacian_4_without_adjust = np.clip(moon_image_laplacian_4,
  0, 255).astype(np.uint8)
19 moon_image_adjust_8 = np.clip(moon_image_laplacian_adjust_8, 0, 255).
  astype(np.uint8)

```

Listing 2: Normalização de imagens antes de aplicar a função de realce Laplaciano.

Conforme algoritmo 2, existe uma fundamental etapa antes do laplaciano ser aplicado, que é a de normalização. A imagem deve ser transformada para float32 para que as operações de realce sejam devidamente calculadas e garantirmos que nenhum truncamento seja realizado durante os cálculos. A seguir, submetemos a imagem função *laplacian_numpy* para cada kernel e então temos 2 imagens realçadas usando a técnica do laplaciano discreto. Conforme Gonzalez e Woods [Gonzalez, 2009], para obtermos uma imagem aguçada ou com o laplaciano ajustado basta subtrairmos a imagem original do realce laplaciano assim como descrito algoritmo 2, por fim, para termos garantia de uma imagem na escala de cinza, realizamos uma operação no NumPY (*np.clip*) que limita os pixels da imagem a estarem em um limar ([0, 255]) e, em seguida, convertemos para a escala de cinza.

2 Resultados

Esta seção apresenta os resultados obtidos a partir da aplicação do operador Laplaciano para realce de imagens, conforme a formulação clássica discutida por Gonzalez e Woods [Gonzalez, 2009]. O trabalho teve como objetivo avaliar os efeitos do filtro Laplaciano, com e sem ajuste, sobre a nitidez e o contraste de uma imagem original, além de comparar duas configurações de kernel.

Os resultados obtidos de cada etapa podem ser observados na Figura 1. A imagem (a) representa o original. Em (b), observa-se o resultado do realce Laplaciano sem ajuste, o qual evidencia um aumento expressivo das bordas, mas também uma amplificação perceptível do ruído em regiões homogêneas. Em (c), o realce Laplaciano com ajuste demonstra um melhor equilíbrio entre a nitidez das bordas e a preservação das áreas uniformes, resultado do processo de adição ponderada da imagem original ao Laplaciano.

As imagens (d) e (e) apresentam resultados com duas máscaras Laplacianas distintas, ambas de dimensão 3×3 . A máscara com o coeficiente central igual a -4 (d) produz um realce mais suave, enquanto a de coeficiente -8 (e) gera um contraste mais agressivo e realce mais intenso das bordas. A diferença entre os dois resultados evidencia a influência

direta da escolha da máscara sobre o grau de nitidez e sobre o surgimento de halos indesejados.

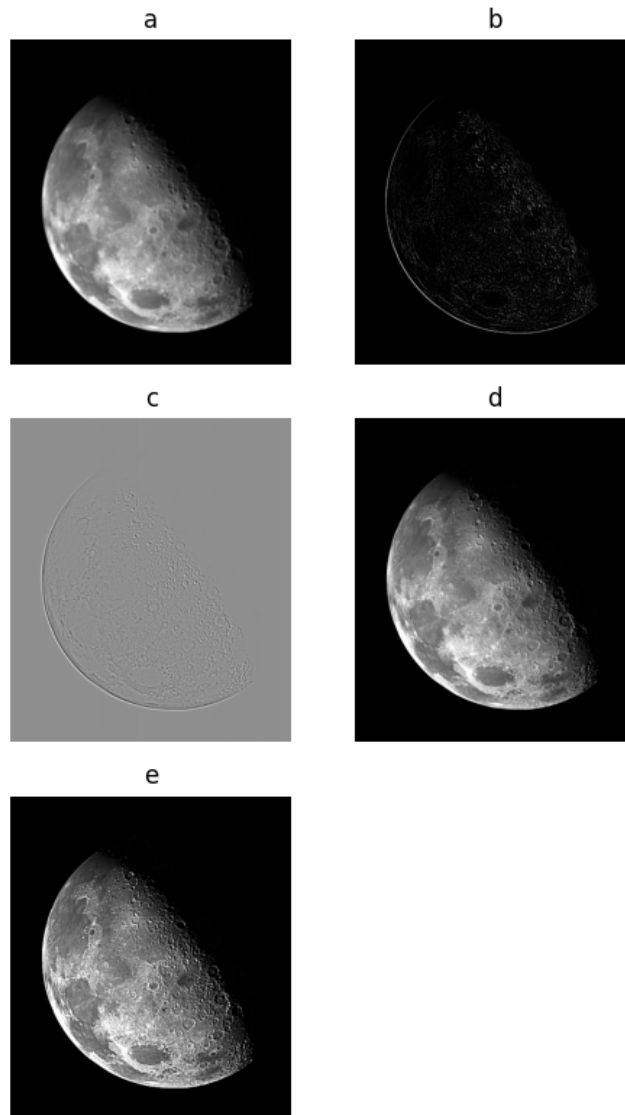


Figura 1: (a) Imagem original; (b) Realce Laplaciano sem ajuste. (c) Realce Laplaciano com ajuste; (d) Imagem aguçada utilizando a máscara 3x3 (com o pixel central -4); (e) Resultado da utilizando a máscara 3x3 (com o pixel central -8).

De modo geral, os resultados confirmam o comportamento esperado do filtro Laplaciano que a de ressaltar transições bruscas de intensidade (bordas) e, conseqüentemente, melhorar a percepção visual de detalhes finos.

3 Conclusão

O estudo do realce Laplaciano demonstrou que esta técnica é altamente eficaz para enfatizar detalhes e bordas em imagens digitais. No entanto, a aplicação direta do operador tende a amplificar ruídos, o que pode comprometer a qualidade visual global quando a imagem contém regiões homogêneas ou pouco contrastadas.

A utilização de um ajuste ou recombinação da imagem original com a sua versão Laplaciana se mostrou fundamental para obter resultados mais equilibrados e de alta qualidade, preservando as texturas e evitando artefatos visuais excessivos. Além disso, a escolha da máscara influencia significativamente o grau de realce, devendo ser definida de acordo com o objetivo do processamento (ênfase em detalhes finos ou em bordas mais marcadas).

Conclui-se, portanto, que o operador Laplaciano é uma ferramenta poderosa para o realce de imagens, mas que sua aplicação requer controle cuidadoso dos parâmetros e, idealmente, deve ser precedida de filtragem para redução de ruído. O comportamento observado está em plena concordância com os fundamentos teóricos apresentados por Gonzalez e Woods [Gonzalez, 2009].

Referências

[Gonzalez, 2009] Gonzalez, R. C. (2009). *Digital image processing*. Pearson education india.

[Rosenfeld, 1976] Rosenfeld, A. (1976). *Digital picture processing*. Academic press.